SRI SHAKTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution
Coimbatore - 62.
POWERING THE YOUTH
EMPOWERING THE NATION
NBA NATIONAL BOARD OF ACCREDITATION For CSE,MECH,EEE, IT
ACCREDITED WITH GRADE A NAAC

MERN PROJECT

NUTRI AI APP

DONE BY:
PRATHEKAA M
III – CSE C

ABSTRACT

NUTRI AI is a MERN-stack-based web application that uses Artificial Intelligence to provide personalized diet and fitness guidance based on individual user requirements. The system collects user information such as height, weight, fitness goal, and other relevant inputs to generate an AI-based diet chart designed to support either weight loss or weight gain. It also provides an estimated daily water intake to help users maintain proper hydration.

One of the key features of NUTRI AI is its AI-powered food analysis system. Users can upload a photo of a food item, and the system analyzes the image to identify the food and provide an estimated calorie value. Based on the user's fitness goal and dietary requirements, the application can indicate whether the food is suitable to consume at that time. The system provides an easy-to-understand interface that helps users monitor their food choices and follow their personalized diet plan.

The application is developed using the MERN stack, consisting of MongoDB, Express.js, React.js, and Node.js, with AI services integrated for personalized diet generation and food image analysis. NUTRI AI aims to make diet management more accessible, personalized, and convenient by combining web technologies with AI-based recommendations in a single platform.

TECH STACK

Frontend

Nutri AI's frontend is built using React.js, providing a component-based architecture for creating an interactive and responsive nutrition management interface. React is used to develop different views of the application, including the user profile, personalized diet chart, water-intake tracking, food calorie analysis, and fitness goal sections. The frontend uses reusable components so that features such as diet cards, calorie information, food-upload interfaces, and progress indicators can be developed and maintained independently.

The application provides an interactive interface where users can enter details such as height, weight, and fitness goal. Based on these inputs, the system displays an AI-generated diet plan designed according to the user's selected goal, such as weight loss or weight gain. Users can also upload food images through the frontend to obtain calorie-related information and determine whether the food is suitable according to their current diet plan.

Backend

The backend of Nutri AI is developed using **Node.js and Express.js**, providing the REST API layer between the React frontend and MongoDB database. The backend manages user information, fitness goals, diet-plan requests, food-analysis requests, and other application-related operations.

Node.js provides the runtime environment for the server-side application, while Express.js handles HTTP requests, routing, and API communication. The backend also acts as the communication layer between the frontend and AI services used for generating personalized diet recommendations and analyzing uploaded food images.

Database

Nutri AI uses **MongoDB** as its primary database for storing application data. MongoDB stores user profiles, height and weight information, fitness goals, diet-related data, water-intake information, and other user-specific records.

Mongoose is used as the Object Data Modeling (ODM) layer between Node.js and MongoDB. It provides schemas and models that define the structure of the application's data and simplifies database operations through the Express.js backend.

API Integration

Artificial Intelligence is one of the core components of Nutri AI. The system uses AI to generate personalized diet recommendations based on user-provided information such as height, weight, and whether the user wants to lose or gain weight.

The AI component also supports food image analysis. Users can upload a photograph of a food item, which is analyzed to identify the food and provide an estimated calorie value. The system can then compare the food information with the user's selected fitness goal and provide a suitable recommendation.

- **REST API Integration** – Connects the React frontend with the Node.js and Express.js backend to securely exchange user and nutrition-related data.
- **AI API Integration** – Connects the backend with AI services to generate personalized diet charts based on height, weight, and fitness goals.
- **Food Image Analysis API** – Sends uploaded food images to an AI-based image analysis service to identify food items and estimate their calorie content.

MERN Architecture

Nutri AI follows the MERN stack architecture, consisting of four primary technologies:

- MongoDB – Stores user profiles, fitness information, diet data, and application records.
- Express.js – Provides the REST API and server-side routing.
- React.js – Builds the interactive frontend and user interfaces.
- Node.js – Provides the server-side JavaScript runtime.

The overall flow of the system is:

React Frontend → Express.js API → Node.js Backend → Mongoose → MongoDB

AI services are integrated with the backend to generate personalized diet charts and analyze food images.

This architecture provides a clear separation between the presentation layer, application logic, database layer, and AI functionality, making Nutri AI easier to develop, maintain, and extend.

MODULES

Module 1: User Authentication & Management

This module handles user registration, login, and account management within the Nutri AI application. Users can create an account and log in to access their personalized nutrition information and diet plans. Authentication is handled through the backend, with user information stored securely in MongoDB.
The module provides the foundation for associating nutrition-related activities with individual users. Once authenticated, users can enter their personal information such as height, weight, and fitness goals. The backend manages user-related requests through Express.js routes and Mongoose models, while the React frontend provides the corresponding registration and login interfaces.

Module 2: User Profile & Fitness Goal Management

This module collects and manages the user's basic fitness information required for personalized nutrition recommendations. Users can provide details such as height, weight, and their desired fitness goal, including weight loss or weight gain.
The entered information is stored in MongoDB and can be updated whenever the user's requirements change. The React frontend provides forms and interfaces for entering and modifying the user's fitness information.
The collected profile information acts as the primary input for the AI-based recommendation system. It allows Nutri AI to generate diet recommendations according to the individual user's requirements rather than providing the same diet plan to every user.

Module 3: AI-Based Personalized Diet Generation

The AI-Based Personalized Diet Generation module is one of the core modules of Nutri AI. It generates a personalized diet chart based on the user's height, weight, fitness goal, and other provided information.
The user's information is sent from the React frontend to the backend through REST APIs. The backend communicates with the integrated AI service, which processes the input and generates suitable dietary recommendations.
The generated diet chart is then returned to the frontend and displayed in an easy-to-understand format. Users can view recommended meals and food choices according to their selected goal, such as weight loss or weight gain.
This module reduces the need for users to manually create a diet plan and provides personalized recommendations through AI integration.

Module 4: Food Image & Calorie Analysis

The Food Image & Calorie Analysis module allows users to upload photographs of food items through the Nutri AI application. The uploaded image is processed using an AI-based image analysis service.

The system analyzes the image to identify the food item and provides an estimated calorie value. The analysis result is returned to the application through the backend API and displayed on the React frontend.

This module helps users understand the approximate nutritional value of the food they are planning to consume. It provides a convenient alternative to manually searching for individual food items and their calorie information.

Module 5: Food Recommendation & Diet Compatibility

This module evaluates the analyzed food information in relation to the user's fitness goal and personalized diet plan

The application provides an easy-to-understand indication of whether the food can be included in the user's current diet plan. The recommendation is presented through the frontend along with the available calorie information.

This module connects the AI food analysis functionality with the personalized diet system, allowing users to make more informed food choices according to their selected fitness objective.

Module 6: Water Intake Management

The Water Intake Management module provides users with an estimated daily water intake target based on their personal information and fitness requirements.

The recommended water intake is displayed through the Nutri AI dashboard so that users can easily monitor their daily hydration target. The frontend provides visual indicators or progress information to make the hydration information easier to understand.

Module 7: Nutrition Dashboard & Progress Tracking

The Nutrition Dashboard provides users with a centralized view of their personalized nutrition information. It brings together the generated diet chart, water-intake information, fitness goal, and other relevant user data.

The dashboard retrieves the required information through backend APIs and presents it using interactive React components. Users can view their current diet recommendations and food-analysis results from a single interface.

The module also provides a convenient way for users to monitor their nutrition-related activities and review their personalized information.

Module 8: API Integration & AI Communication

The API Integration module acts as the communication layer between the React frontend, Node.js/Express.js backend, database, and external AI services.

REST APIs are used to send user information from the frontend to the backend and retrieve generated diet plans, food-analysis results, and other nutrition-related information.

The backend communicates with AI APIs for personalized diet generation and food image analysis. The responses from these services are processed by the backend before being delivered to the React frontend.

This module provides a structured connection between the different components of Nutri AI and allows the application to integrate AI functionality into the MERN architecture.

FRONTEND CODE

```
import express from "express";
import path from "path";
import { createServer as createViteServer } from "vite";
import { GoogleGenAI, Type } from "@google/genai";
import dotenv from "dotenv";

dotenv.config();

const app = express();
const PORT = 3000;

app.use(express.json({ limit: '10mb' }));

const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY,
  httpOptions: {
    headers: {
      'User-Agent': 'aistudio-build',
    }
  }
});

app.post("/api/generate-diet", async (req, res) => {
  try {
    const {
      name, age, gender, height, weight,
      activityLevel, healthGoal,
      foodPreference, allergies, foodsToAvoid,
      mealsPerDay, preferredCuisine, budget, waterIntake,
      bmi, bmr, calorieRequirement,
      proteinRequirement, carbsRequirement, fatRequirement
    } = req.body;
```

```
const prompt = `
Generate a personalized 7-day diet plan for ${name}, a ${age} year old ${gender}.
You MUST provide exactly 7 days in the "days" array (dayNumber 1 through 7). Do not stop at Day 1. Each of the 7 days must have its own breakfast, morningSnack, lunch, eveningSnack, and dinner.
Height: ${height} cm, Weight: ${weight} kg.
Activity Level: ${activityLevel}.
Health Goal: ${healthGoal}.
Food Preference: ${foodPreference}.
Allergies: ${allergies || 'None'}.
Foods to avoid: ${foodsToAvoid || 'None'}.
Meals per day: ${mealsPerDay || 'Standard 3 meals + 2 snacks'}.
Preferred Cuisine: ${preferredCuisine || 'Any'}.
Budget: ${budget || 'Average'}.

Nutritional Targets:
Daily Calories: ${calorieRequirement} kcal
Protein: ${proteinRequirement} g
Carbs: ${carbsRequirement} g
Fat: ${fatRequirement} g

Ensure the plan meets these exact macro targets as closely as possible, and provide realistic food options.
Also ensure you cover micronutrients properly (Iron, Calcium, Vit A, Vit B1, Vit B2, Vit B3, Vit B6, Vit B12, Vit C, Vit D, Vit E, Vit K, Folate, Magnesium, Zinc, Potassium).
If the generated diet does not adequately cover a nutrient, you MUST provide suitable food sources to improve it in the micronutrientSuggestions array.
Do not suggest foods from their allergies or 'foods to avoid' list.
Provide realistic Indian food examples (rice, idli, dosa, dal, paneer, chicken, etc.) if Indian cuisine is preferred, else diverse healthy options.
`;

const mealSchema = {
```

```
  type: Type.OBJECT,
  properties: {
    foodName: { type: Type.STRING },
    portionSize: { type: Type.STRING },
    calories: { type: Type.NUMBER },
    protein: { type: Type.NUMBER },
    carbs: { type: Type.NUMBER },
    fat: { type: Type.NUMBER },
  },
  required: ["foodName", "portionSize", "calories", "protein", "carbs",
"fat"]
};

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash-lite",
      contents: prompt,
      config: {
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            days: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  dayNumber: { type: Type.INTEGER },
                  meals: {
                    type: Type.OBJECT,
                    properties: {
                      breakfast: mealSchema,
                      morningSnack: mealSchema,
                      lunch: mealSchema,
                      eveningSnack: mealSchema,
                      dinner: mealSchema,
```

```
            },
            required: ["breakfast", "morningSnack", "lunch",
"eveningSnack", "dinner"]
          },
          summary: {
            type: Type.OBJECT,
            properties: {
              calories: { type: Type.NUMBER },
              protein: { type: Type.NUMBER },
              carbs: { type: Type.NUMBER },
              fat: { type: Type.NUMBER },
              micronutrients: {
                type: Type.OBJECT,
                properties: {
                  ironCoveragePct: { type: Type.NUMBER },
                  calciumCoveragePct: { type: Type.NUMBER },
                  vitACoveragePct: { type: Type.NUMBER },
                  vitB12CoveragePct: { type: Type.NUMBER },
                  vitCCoveragePct: { type: Type.NUMBER },
                  vitDCoveragePct: { type: Type.NUMBER },
                  magnesiumCoveragePct: { type: Type.NUMBER },
                  zincCoveragePct: { type: Type.NUMBER },
                  potassiumCoveragePct: { type: Type.NUMBER },
                },
                required: ["ironCoveragePct", "calciumCoveragePct",
"vitACoveragePct", "vitB12CoveragePct", "vitCCoveragePct",
"vitDCoveragePct", "magnesiumCoveragePct", "zincCoveragePct",
"potassiumCoveragePct"]
              }
            },
            required: ["calories", "protein", "carbs", "fat",
"micronutrients"]
          }
        },
        required: ["dayNumber", "meals", "summary"]
      }
```

```
        },
        micronutrientSuggestions: {
          type: Type.ARRAY,
          items: {
            type: Type.OBJECT,
            properties: {
              nutrient: { type: Type.STRING },
              suggestion: { type: Type.STRING }
            }
          }
        }
      }
    }
  });

  const data = JSON.parse(response.text?.trim() || "{}");
  res.json(data);
 } catch (error) {
  console.error("Error generating diet:", error);
  res.status(500).json({ error: "Failed to generate diet" });
 }
});

app.post("/api/scan-food", async (req, res) => {
 try {
  const { image, targets, user } = req.body; // image is base64 data url

  if (!image) {
   return res.status(400).json({ error: "Image is required" });
  }

  const base64Data = image.split(',')[1];
  const mimeType = image.split(';')[0].split(':')[1];
```

BACKEND CODE

```
import { useState } from 'react';
import InputForm from './components/InputForm';
import Dashboard from './components/Dashboard';
import { UserProfile, NutritionTargets, DietPlan } from './types';
import { calculateNutrition } from './utils/nutrition';

export default function App() {
  const [userProfile, setUserProfile] = useState<UserProfile | null>(null);
  const [nutritionTargets, setNutritionTargets] = useState<NutritionTargets | null>(null);
  const [dietPlan, setDietPlan] = useState<DietPlan | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const generateDiet = async (profile: UserProfile, targets: NutritionTargets) => {
    setIsLoading(true);
    setError(null);
    try {
      const response = await fetch('/api/generate-diet', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          ...profile,
          ...targets
        })
      });

      if (!response.ok) {
        throw new Error('Failed to generate diet plan');
      }

      const data = await response.json();
      setDietPlan(data);
      setUserProfile(profile);
      setNutritionTargets(targets);
    } catch (err: any) {
      console.error(err);
      setError(err.message || 'An error occurred while generating the diet plan.');
    } finally {
      setIsLoading(false);
    }
  };

  const handleFormSubmit = (data: UserProfile) => {
    const targets = calculateNutrition(data);
    generateDiet(data, targets);
  };
```

```
const handleRegenerate = () => {
    if (userProfile && nutritionTargets) {
      generateDiet(userProfile, nutritionTargets);
    }
  };

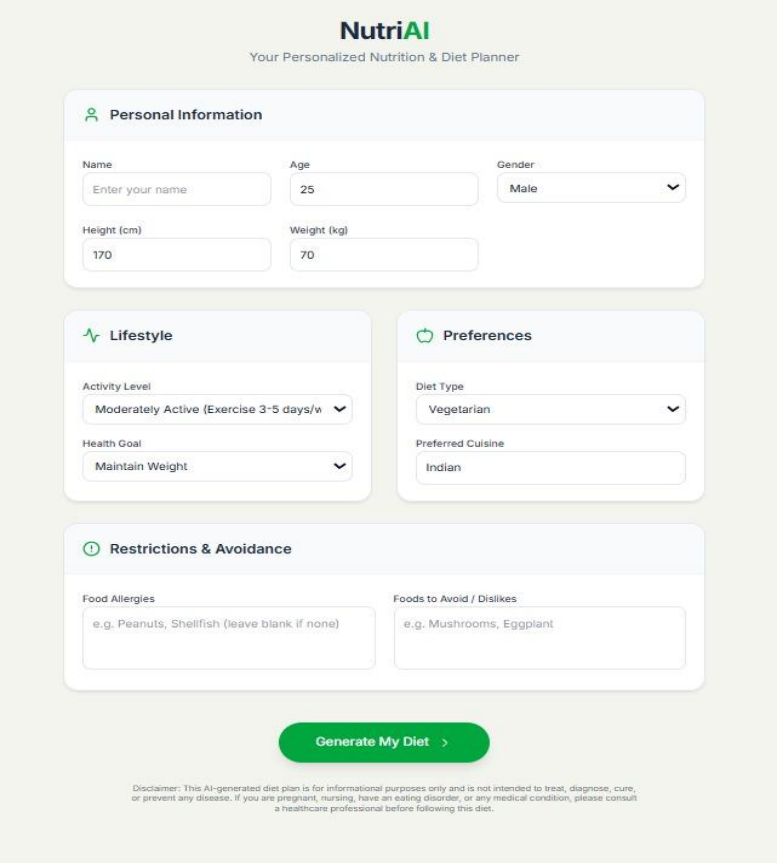
  const handleBack = () => {
    setDietPlan(null);
  };

  return (
    <div className="min-h-screen bg-organic font-sans text-slate-900">
      <main className="py-8 px-4 h-full flex flex-col">
        {error && (
          <div className="max-w-4xl mx-auto mb-6 p-4 bg-red-50 border border-red-200 text-
red-700 rounded-lg">
            {error}
          </div>
        )}

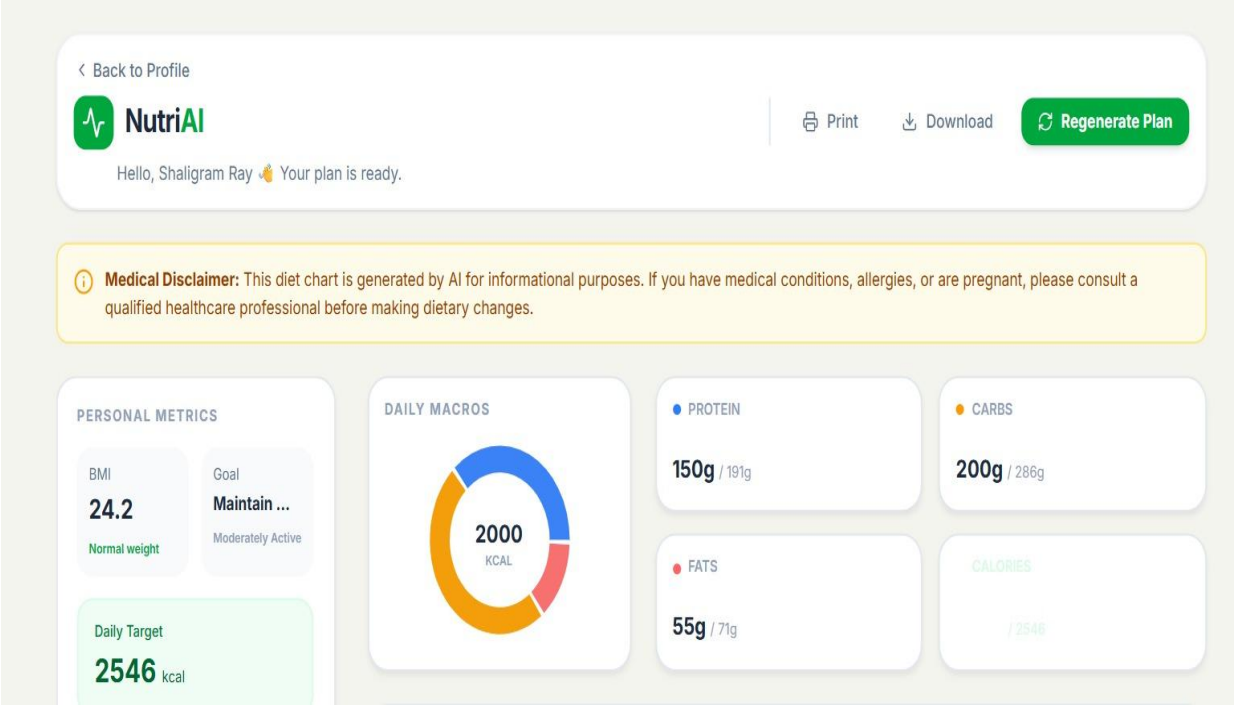
        {!dietPlan ? (
          <InputForm onSubmit={handleFormSubmit} isLoading={isLoading} />
        ) : (
          <Dashboard
            user={userProfile!}
            targets={nutritionTargets!}
            dietPlan={dietPlan}
            onRegenerate={handleRegenerate}
            onBack={handleBack}
            isRegenerating={isLoading}
          />
        )}
      </main>
    </div>
  );
}
```

OUTPUT

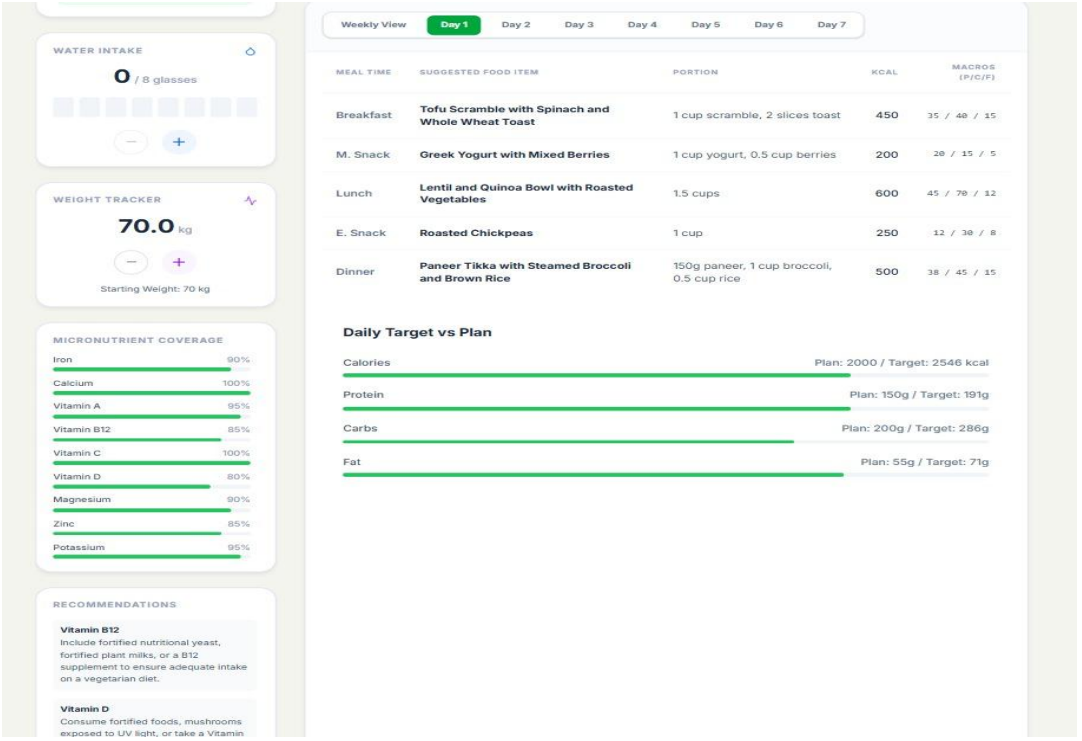
Home Page:



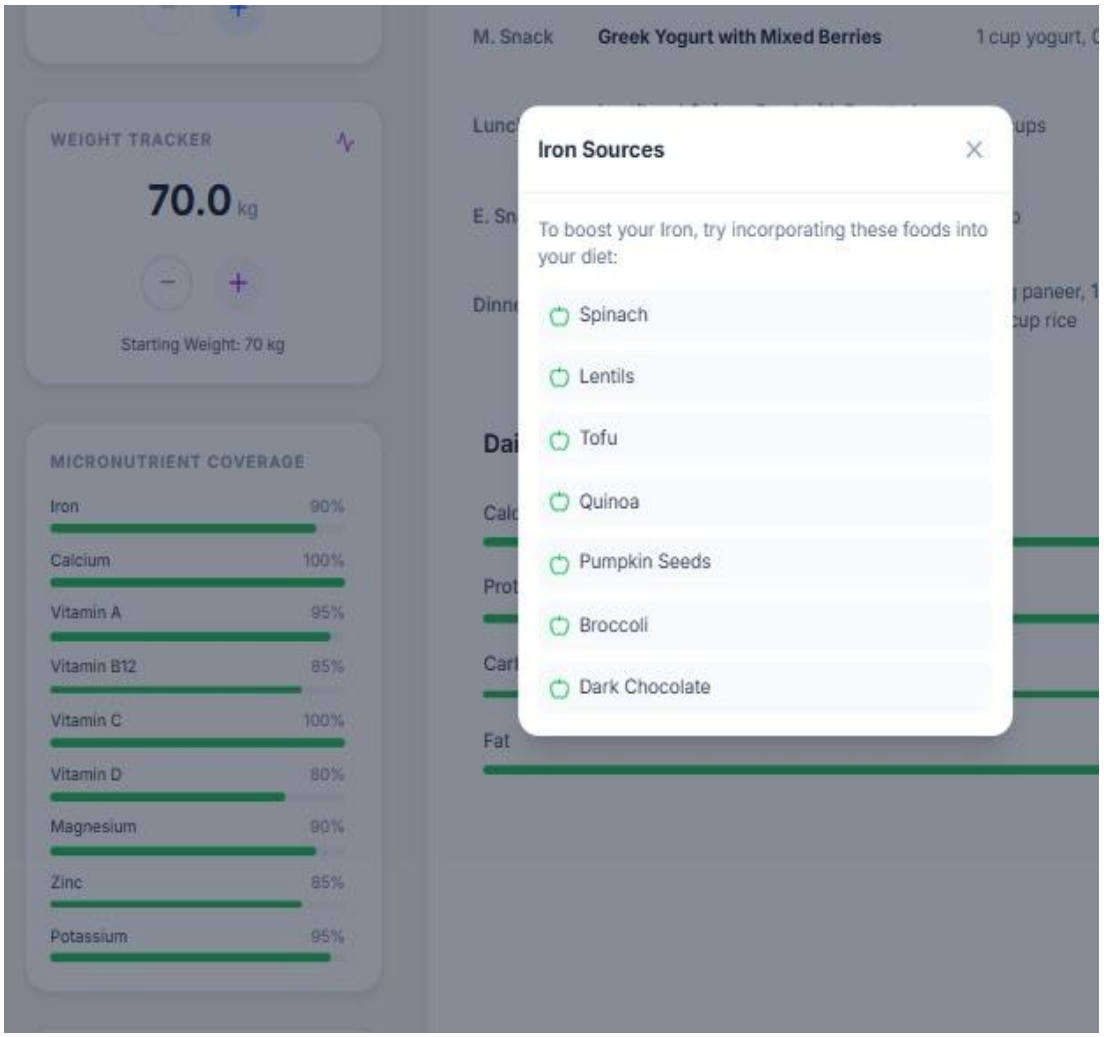
Dashboard:



Daily Tracker Page:



Discover Nutrients:



SRI SHAKTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution
Coimbatore - 62.
POWERING THE YOUTH
EMPOWERING THE NATION
NBA NATIONAL BOARD OF ACCREDITATION For CSE,MECH,EEE, IT
ACCREDITED WITH GRADE A NAAC

MERN PROJECT

NUTRI AI APP

DONE BY:
PREETHI KRISHNA B
III – CSE C

ABSTRACT

NUTRI AI is a MERN-stack-based web application that uses Artificial Intelligence to provide personalized diet and fitness guidance based on individual user requirements. The system collects user information such as height, weight, fitness goal, and other relevant inputs to generate an AI-based diet chart designed to support either weight loss or weight gain. It also provides an estimated daily water intake to help users maintain proper hydration.

One of the key features of NUTRI AI is its AI-powered food analysis system. Users can upload a photo of a food item, and the system analyzes the image to identify the food and provide an estimated calorie value. Based on the user's fitness goal and dietary requirements, the application can indicate whether the food is suitable to consume at that time. The system provides an easy-to-understand interface that helps users monitor their food choices and follow their personalized diet plan.

The application is developed using the MERN stack, consisting of MongoDB, Express.js, React.js, and Node.js, with AI services integrated for personalized diet generation and food image analysis. NUTRI AI aims to make diet management more accessible, personalized, and convenient by combining web technologies with AI-based recommendations in a single platform.

TECH STACK

Frontend

Nutri AI's frontend is built using React.js, providing a component-based architecture for creating an interactive and responsive nutrition management interface. React is used to develop different views of the application, including the user profile, personalized diet chart, water-intake tracking, food calorie analysis, and fitness goal sections. The frontend uses reusable components so that features such as diet cards, calorie information, food-upload interfaces, and progress indicators can be developed and maintained independently.

The application provides an interactive interface where users can enter details such as height, weight, and fitness goal. Based on these inputs, the system displays an AI-generated diet plan designed according to the user's selected goal, such as weight loss or weight gain. Users can also upload food images through the frontend to obtain calorie-related information and determine whether the food is suitable according to their current diet plan.

Backend

The backend of Nutri AI is developed using **Node.js and Express.js**, providing the REST API layer between the React frontend and MongoDB database. The backend manages user information, fitness goals, diet-plan requests, food-analysis requests, and other application-related operations.

Node.js provides the runtime environment for the server-side application, while Express.js handles HTTP requests, routing, and API communication. The backend also acts as the communication layer between the frontend and AI services used for generating personalized diet recommendations and analyzing uploaded food images.

Database

Nutri AI uses **MongoDB** as its primary database for storing application data. MongoDB stores user profiles, height and weight information, fitness goals, diet-related data, water-intake information, and other user-specific records.

Mongoose is used as the Object Data Modeling (ODM) layer between Node.js and MongoDB. It provides schemas and models that define the structure of the application's data and simplifies database operations through the Express.js backend.

API Integration

Artificial Intelligence is one of the core components of Nutri AI. The system uses AI to generate personalized diet recommendations based on user-provided information such as height, weight, and whether the user wants to lose or gain weight.

The AI component also supports food image analysis. Users can upload a photograph of a food item, which is analyzed to identify the food and provide an estimated calorie value. The system can then compare the food information with the user's selected fitness goal and provide a suitable recommendation.

- **REST API Integration** – Connects the React frontend with the Node.js and Express.js backend to securely exchange user and nutrition-related data.
- **AI API Integration** – Connects the backend with AI services to generate personalized diet charts based on height, weight, and fitness goals.
- **Food Image Analysis API** – Sends uploaded food images to an AI-based image analysis service to identify food items and estimate their calorie content.

MERN Architecture

Nutri AI follows the MERN stack architecture, consisting of four primary technologies:

- MongoDB – Stores user profiles, fitness information, diet data, and application records.
- Express.js – Provides the REST API and server-side routing.
- React.js – Builds the interactive frontend and user interfaces.
- Node.js – Provides the server-side JavaScript runtime.

The overall flow of the system is:

React Frontend → Express.js API → Node.js Backend → Mongoose → MongoDB

AI services are integrated with the backend to generate personalized diet charts and analyze food images.

This architecture provides a clear separation between the presentation layer, application logic, database layer, and AI functionality, making Nutri AI easier to develop, maintain, and extend.

MODULES

Module 1: User Authentication & Management

This module handles user registration, login, and account management within the Nutri AI application. Users can create an account and log in to access their personalized nutrition information and diet plans. Authentication is handled through the backend, with user information stored securely in MongoDB.
The module provides the foundation for associating nutrition-related activities with individual users. Once authenticated, users can enter their personal information such as height, weight, and fitness goals. The backend manages user-related requests through Express.js routes and Mongoose models, while the React frontend provides the corresponding registration and login interfaces.

Module 2: User Profile & Fitness Goal Management

This module collects and manages the user's basic fitness information required for personalized nutrition recommendations. Users can provide details such as height, weight, and their desired fitness goal, including weight loss or weight gain.
The entered information is stored in MongoDB and can be updated whenever the user's requirements change. The React frontend provides forms and interfaces for entering and modifying the user's fitness information.
The collected profile information acts as the primary input for the AI-based recommendation system. It allows Nutri AI to generate diet recommendations according to the individual user's requirements rather than providing the same diet plan to every user.

Module 3: AI-Based Personalized Diet Generation

The AI-Based Personalized Diet Generation module is one of the core modules of Nutri AI. It generates a personalized diet chart based on the user's height, weight, fitness goal, and other provided information.
The user's information is sent from the React frontend to the backend through REST APIs. The backend communicates with the integrated AI service, which processes the input and generates suitable dietary recommendations.
The generated diet chart is then returned to the frontend and displayed in an easy-to-understand format. Users can view recommended meals and food choices according to their selected goal, such as weight loss or weight gain.
This module reduces the need for users to manually create a diet plan and provides personalized recommendations through AI integration.

Module 4: Food Image & Calorie Analysis

The Food Image & Calorie Analysis module allows users to upload photographs of food items through the Nutri AI application. The uploaded image is processed using an AI-based image analysis service.

The system analyzes the image to identify the food item and provides an estimated calorie value. The analysis result is returned to the application through the backend API and displayed on the React frontend.

This module helps users understand the approximate nutritional value of the food they are planning to consume. It provides a convenient alternative to manually searching for individual food items and their calorie information.

Module 5: Food Recommendation & Diet Compatibility

This module evaluates the analyzed food information in relation to the user's fitness goal and personalized diet plan

The application provides an easy-to-understand indication of whether the food can be included in the user's current diet plan. The recommendation is presented through the frontend along with the available calorie information.

This module connects the AI food analysis functionality with the personalized diet system, allowing users to make more informed food choices according to their selected fitness objective.

Module 6: Water Intake Management

The Water Intake Management module provides users with an estimated daily water intake target based on their personal information and fitness requirements.

The recommended water intake is displayed through the Nutri AI dashboard so that users can easily monitor their daily hydration target. The frontend provides visual indicators or progress information to make the hydration information easier to understand.

Module 7: Nutrition Dashboard & Progress Tracking

The Nutrition Dashboard provides users with a centralized view of their personalized nutrition information. It brings together the generated diet chart, water-intake information, fitness goal, and other relevant user data.

The dashboard retrieves the required information through backend APIs and presents it using interactive React components. Users can view their current diet recommendations and food-analysis results from a single interface.

The module also provides a convenient way for users to monitor their nutrition-related activities and review their personalized information.

Module 8: API Integration & AI Communication

The API Integration module acts as the communication layer between the React frontend, Node.js/Express.js backend, database, and external AI services.

REST APIs are used to send user information from the frontend to the backend and retrieve generated diet plans, food-analysis results, and other nutrition-related information.

The backend communicates with AI APIs for personalized diet generation and food image analysis. The responses from these services are processed by the backend before being delivered to the React frontend.

This module provides a structured connection between the different components of Nutri AI and allows the application to integrate AI functionality into the MERN architecture.

FRONTEND CODE

```
import express from "express";
import path from "path";
import { createServer as createViteServer } from "vite";
import { GoogleGenAI, Type } from "@google/genai";
import dotenv from "dotenv";

dotenv.config();

const app = express();
const PORT = 3000;

app.use(express.json({ limit: '10mb' }));

const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY,
  httpOptions: {
    headers: {
      'User-Agent': 'aistudio-build',
    }
  }
});

app.post("/api/generate-diet", async (req, res) => {
  try {
    const {
      name, age, gender, height, weight,
      activityLevel, healthGoal,
      foodPreference, allergies, foodsToAvoid,
      mealsPerDay, preferredCuisine, budget, waterIntake,
      bmi, bmr, calorieRequirement,
      proteinRequirement, carbsRequirement, fatRequirement
    } = req.body;
```

```
const prompt = `
Generate a personalized 7-day diet plan for ${name}, a ${age} year old ${gender}.
You MUST provide exactly 7 days in the "days" array (dayNumber 1 through 7). Do not stop at Day 1. Each of the 7 days must have its own breakfast, morningSnack, lunch, eveningSnack, and dinner.
Height: ${height} cm, Weight: ${weight} kg.
Activity Level: ${activityLevel}.
Health Goal: ${healthGoal}.
Food Preference: ${foodPreference}.
Allergies: ${allergies || 'None'}.
Foods to avoid: ${foodsToAvoid || 'None'}.
Meals per day: ${mealsPerDay || 'Standard 3 meals + 2 snacks'}.
Preferred Cuisine: ${preferredCuisine || 'Any'}.
Budget: ${budget || 'Average'}.

Nutritional Targets:
Daily Calories: ${calorieRequirement} kcal
Protein: ${proteinRequirement} g
Carbs: ${carbsRequirement} g
Fat: ${fatRequirement} g

Ensure the plan meets these exact macro targets as closely as possible, and provide realistic food options.
Also ensure you cover micronutrients properly (Iron, Calcium, Vit A, Vit B1, Vit B2, Vit B3, Vit B6, Vit B12, Vit C, Vit D, Vit E, Vit K, Folate, Magnesium, Zinc, Potassium).
If the generated diet does not adequately cover a nutrient, you MUST provide suitable food sources to improve it in the micronutrientSuggestions array.
Do not suggest foods from their allergies or 'foods to avoid' list.
Provide realistic Indian food examples (rice, idli, dosa, dal, paneer, chicken, etc.) if Indian cuisine is preferred, else diverse healthy options.
`;

const mealSchema = {
```

```
  type: Type.OBJECT,
  properties: {
    foodName: { type: Type.STRING },
    portionSize: { type: Type.STRING },
    calories: { type: Type.NUMBER },
    protein: { type: Type.NUMBER },
    carbs: { type: Type.NUMBER },
    fat: { type: Type.NUMBER },
  },
  required: ["foodName", "portionSize", "calories", "protein", "carbs",
"fat"]
};

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash-lite",
      contents: prompt,
      config: {
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            days: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  dayNumber: { type: Type.INTEGER },
                  meals: {
                    type: Type.OBJECT,
                    properties: {
                      breakfast: mealSchema,
                      morningSnack: mealSchema,
                      lunch: mealSchema,
                      eveningSnack: mealSchema,
                      dinner: mealSchema,
```

```
            },
            required: ["breakfast", "morningSnack", "lunch",
"eveningSnack", "dinner"]
          },
          summary: {
            type: Type.OBJECT,
            properties: {
              calories: { type: Type.NUMBER },
              protein: { type: Type.NUMBER },
              carbs: { type: Type.NUMBER },
              fat: { type: Type.NUMBER },
              micronutrients: {
                type: Type.OBJECT,
                properties: {
                  ironCoveragePct: { type: Type.NUMBER },
                  calciumCoveragePct: { type: Type.NUMBER },
                  vitACoveragePct: { type: Type.NUMBER },
                  vitB12CoveragePct: { type: Type.NUMBER },
                  vitCCoveragePct: { type: Type.NUMBER },
                  vitDCoveragePct: { type: Type.NUMBER },
                  magnesiumCoveragePct: { type: Type.NUMBER },
                  zincCoveragePct: { type: Type.NUMBER },
                  potassiumCoveragePct: { type: Type.NUMBER },
                },
                required: ["ironCoveragePct", "calciumCoveragePct",
"vitACoveragePct", "vitB12CoveragePct", "vitCCoveragePct",
"vitDCoveragePct", "magnesiumCoveragePct", "zincCoveragePct",
"potassiumCoveragePct"]
              }
            },
            required: ["calories", "protein", "carbs", "fat",
"micronutrients"]
          }
        },
        required: ["dayNumber", "meals", "summary"]
      }
```

```
        },
        micronutrientSuggestions: {
          type: Type.ARRAY,
          items: {
            type: Type.OBJECT,
            properties: {
              nutrient: { type: Type.STRING },
              suggestion: { type: Type.STRING }
            }
          }
        }
      }
    }
  });

  const data = JSON.parse(response.text?.trim() || "{}");
  res.json(data);
 } catch (error) {
  console.error("Error generating diet:", error);
  res.status(500).json({ error: "Failed to generate diet" });
 }
});

app.post("/api/scan-food", async (req, res) => {
 try {
  const { image, targets, user } = req.body; // image is base64 data url

  if (!image) {
   return res.status(400).json({ error: "Image is required" });
  }

  const base64Data = image.split(',')[1];
  const mimeType = image.split(';')[0].split(':')[1];
```

BACKEND CODE

```
import { useState } from 'react';
import InputForm from './components/InputForm';
import Dashboard from './components/Dashboard';
import { UserProfile, NutritionTargets, DietPlan } from './types';
import { calculateNutrition } from './utils/nutrition';

export default function App() {
  const [userProfile, setUserProfile] = useState<UserProfile | null>(null);
  const [nutritionTargets, setNutritionTargets] = useState<NutritionTargets | null>(null);
  const [dietPlan, setDietPlan] = useState<DietPlan | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const generateDiet = async (profile: UserProfile, targets: NutritionTargets) => {
    setIsLoading(true);
    setError(null);
    try {
      const response = await fetch('/api/generate-diet', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          ...profile,
          ...targets
        })
      });

      if (!response.ok) {
        throw new Error('Failed to generate diet plan');
      }

      const data = await response.json();
      setDietPlan(data);
      setUserProfile(profile);
      setNutritionTargets(targets);
    } catch (err: any) {
      console.error(err);
      setError(err.message || 'An error occurred while generating the diet plan.');
    } finally {
      setIsLoading(false);
    }
  };

  const handleFormSubmit = (data: UserProfile) => {
    const targets = calculateNutrition(data);
    generateDiet(data, targets);
  };
```

```
const handleRegenerate = () => {
    if (userProfile && nutritionTargets) {
      generateDiet(userProfile, nutritionTargets);
    }
  };

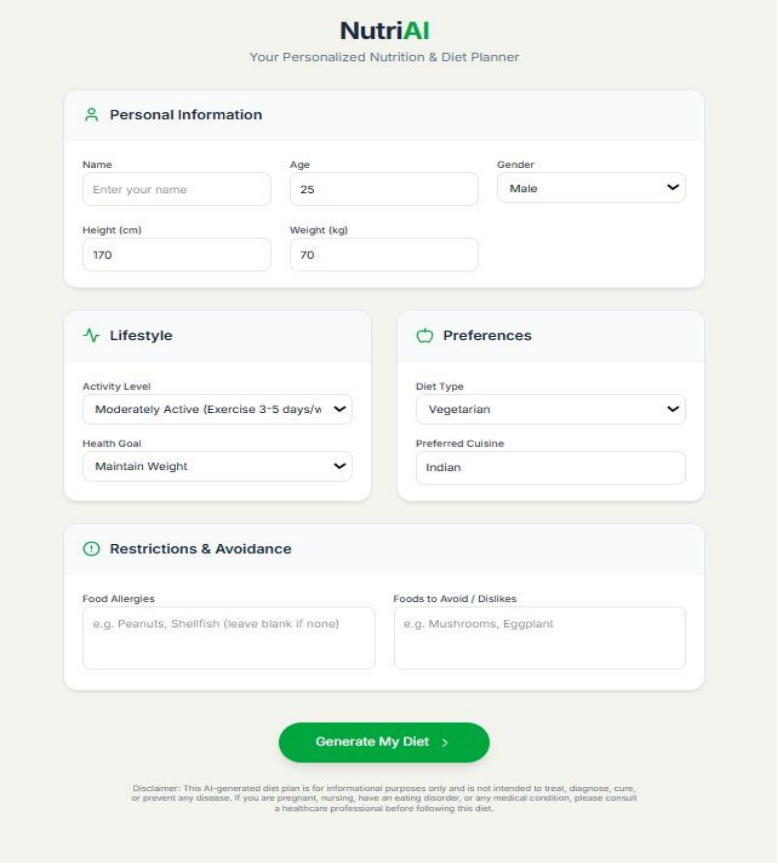
  const handleBack = () => {
    setDietPlan(null);
  };

  return (
    <div className="min-h-screen bg-organic font-sans text-slate-900">
      <main className="py-8 px-4 h-full flex flex-col">
        {error && (
          <div className="max-w-4xl mx-auto mb-6 p-4 bg-red-50 border border-red-200 text-
red-700 rounded-lg">
            {error}
          </div>
        )}

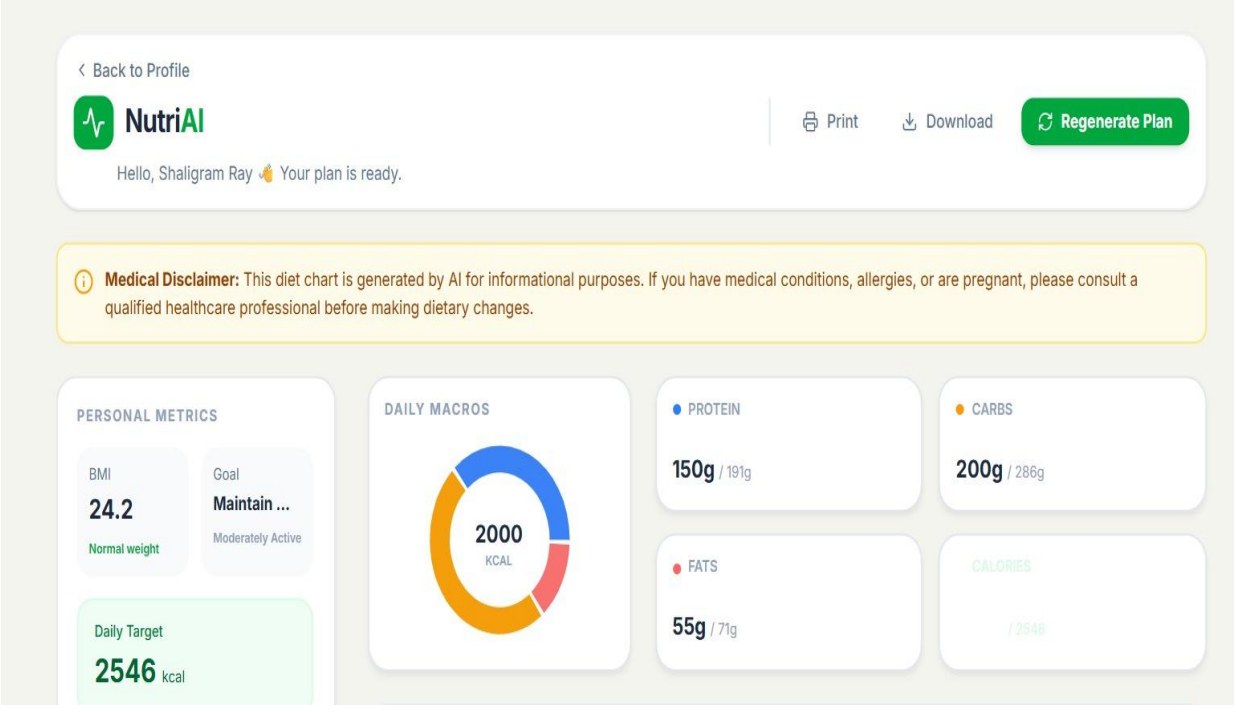
        {!dietPlan ? (
          <InputForm onSubmit={handleFormSubmit} isLoading={isLoading} />
        ) : (
          <Dashboard
            user={userProfile!}
            targets={nutritionTargets!}
            dietPlan={dietPlan}
            onRegenerate={handleRegenerate}
            onBack={handleBack}
            isRegenerating={isLoading}
          />
        )}
      </main>
    </div>
  );
}
```

OUTPUT

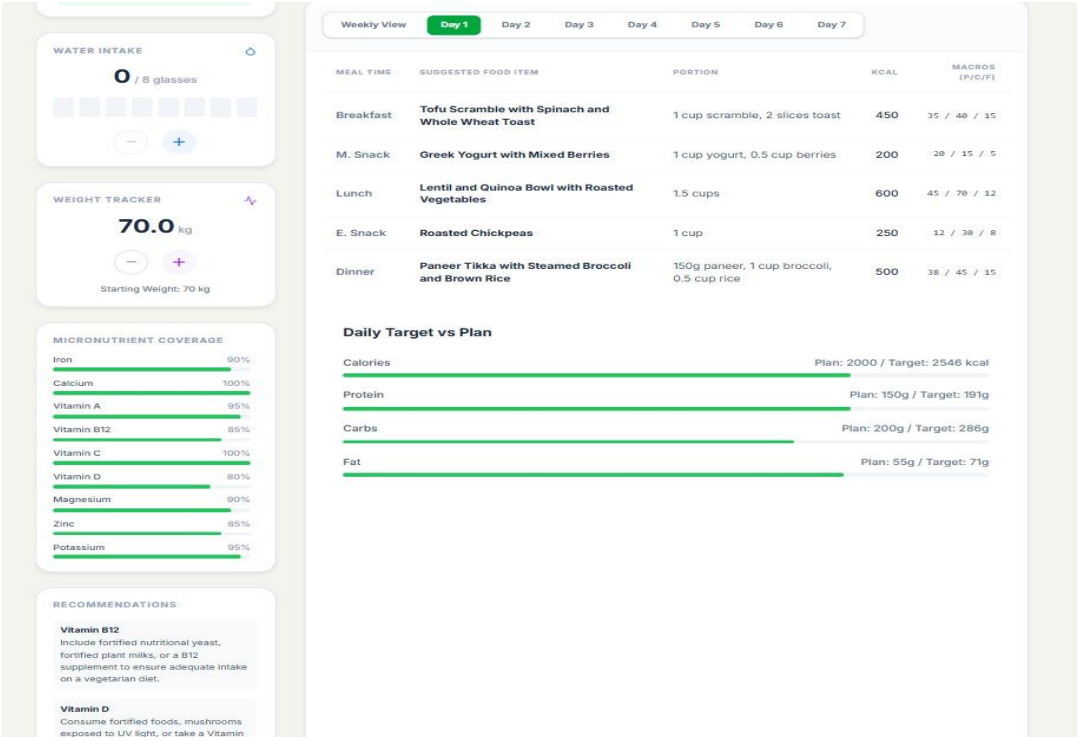
Home Page:



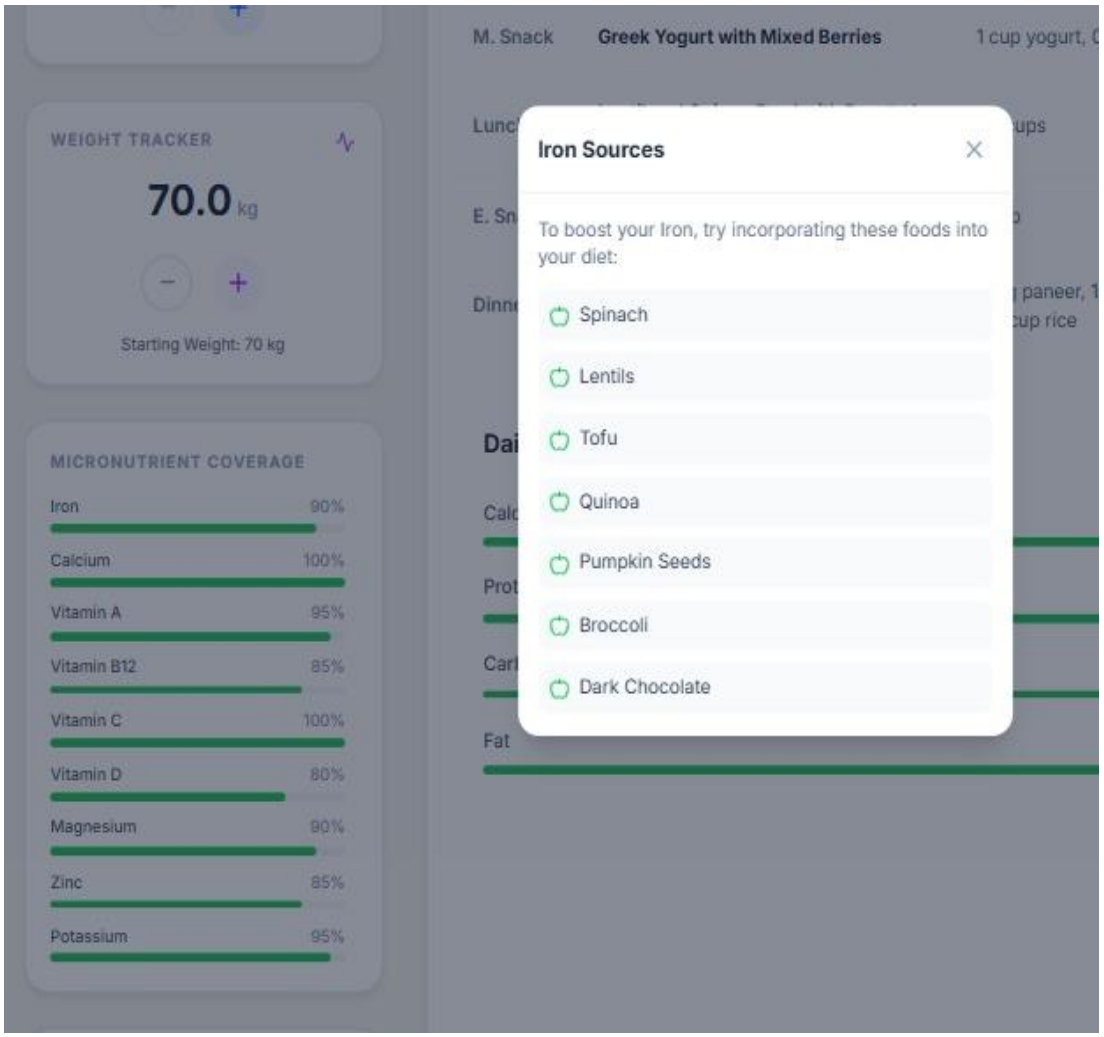
Dashboard:



Daily Tracker Page:



Discover Nutrients:



SRI SHAKTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution
Coimbatore - 62.
POWERING THE YOUTH
EMPOWERING THE NATION
NBA NATIONAL BOARD OF ACCREDITATION For CSE,MECH,EEE, IT
NAAC

MERN PROJECT

POCKET LAWYER
A LEGAL DOCUMENT ASSISTANT

DONE BY:
RADIKAA R
III - CSE - C

ABSTRACT

Pocket Lawyer is a MERN-based legal document assistant designed to help users understand and interact with their legal documents through document upload, document-based question answering, legal topic exploration, and personalized document management. Instead of manually reading lengthy legal documents, users can upload supported files and ask questions based on the content of the selected document.

The system provides a personalized workspace where users can register and log in, upload legal documents, view their uploaded documents, select a document, and ask questions related to its contents. The application focuses on providing answers based on the uploaded document rather than treating every question as a general web search.

The platform is implemented using the MERN stack, with React.js responsible for the interactive frontend, Node.js and Express.js providing the backend REST APIs, and MongoDB serving as the database. Mongoose is used as the object data modeling layer between the backend and MongoDB..

Pocket Lawyer supports document formats including PDF, DOC, DOCX, and TXT, with a maximum upload size of 10 MB. The uploaded document is processed and its text is extracted and stored for document-based question answering.

The system is organized into core modules including User Authentication and Management, Document Upload and Management, Document Text Extraction, Document-Based Question Answering, Legal Topics and Search, Bookmarks and Search History, and User Dashboard and Document Workspace. By combining these modules, Pocket Lawyer provides an integrated platform for users to work with legal documents in a simple and organized manner.

TECH STACK

Frontend

Pocket Lawyer's frontend is built using React.js, providing a component-based architecture for creating an interactive and responsive user interface.

React manages different views of the application, including authentication, dashboard, document upload, document listing, document details, legal topics, and document-based question answering.

The frontend is structured using reusable components so that features such as navigation bars, authentication forms, document cards, upload forms, question panels, answer sections, dashboards, and legal-topic sections can be developed and maintained independently.

The document interface is a central part of the frontend. It allows users to view their uploaded documents, select a document, and interact with the question-answering functionality. The frontend communicates with the backend through REST APIs. User authentication information is used to protect personal documents and user-specific operations.

Backend

The backend of Pocket Lawyer is developed using Node.js and Express.js, providing the REST API layer between the React frontend and MongoDB.

The backend follows a modular structure in which different application responsibilities are handled through separate routes, controllers, models, middleware, and services rather than placing all functionality into a single server file.

The backend manages user authentication, document upload, document retrieval, document processing, question answering, legal topics, bookmarks, and search history.
REST APIs allow the frontend to register and authenticate users, upload documents, retrieve documents, submit questions, retrieve answers, and manage user-specific information.
Node.js provides the runtime environment for the server-side application, while Express.js handles HTTP requests, routing, middleware, and API communication.

This separation between the frontend and backend allows the user interface, business logic, database operations, and document-processing functionality to be developed independently.

Database

Pocket Lawyer uses MongoDB as its primary database for storing application data.

MongoDB stores different types of information required by the application, including:

- User accounts
- Uploaded documents
- Extracted document text
- Document questions and answers
- Legal topics
- Bookmarks
- Search history

Mongoose is used as the object data modeling layer between Node.js and MongoDB. It provides schemas and models that define the structure of application data and simplifies database operations from the Express.js backend.

The database associates documents and questions with their respective users so that each user can maintain a personalized legal document workspace.

The stored document information allows users to access their previously uploaded documents and continue asking questions about them across different sessions.

Document Processing Engine

Document processing is one of the core technical components of Pocket Lawyer.

When a supported legal document is uploaded, the backend processes the file and extracts its readable text. The extracted text is associated with the uploaded document and stored for later retrieval.

Pocket Lawyer supports the following document formats:

- PDF
- DOC
- DOCX
- TXT

The maximum supported upload size is **10 MB**.

The extracted document text becomes the primary source for the document-based question-answering system.

This approach allows the application to process the content of a user's own legal document instead of requiring the user to manually search through a lengthy document.

Document-Based Question Answering

The Document-Based Question Answering module is one of the main features of Pocket Lawyer.

Users can select one of their uploaded documents and enter a question related to its contents.

The backend receives the selected document and question, retrieves the relevant content from the stored document text, and identifies passages that are related to the question.

The relevant passages are then used to produce an answer based on the selected document.

The system is designed so that the question-answering process remains focused on the selected document.

The result can contain:

- User question
- Relevant document information
- Generated answer
- Associated document
- Question history
-

The question and answer information can be stored in MongoDB so that users can review their previous interactions.

API Layer

The Pocket Lawyer REST API is organized around the major resources and functions of the application.

The backend provides endpoints for:

- User authentication
- Document upload
- Document retrieval
- Document-based questions
- Legal topics
- Bookmarks
- Search history

The API layer acts as the communication bridge between the React frontend and MongoDB.

When a user performs an operation through the interface, the frontend sends the appropriate request to the Express.js server.

The backend processes the request, verifies authentication and authorization, performs the required business logic, interacts with MongoDB through Mongoose, and returns the required response to the frontend.

This architecture keeps database operations and application logic on the server while allowing the React frontend to focus primarily on presentation and user interaction.

Authentication, document access, question processing, and database operations remain protected within the backend.

MERN Architecture

Pocket Lawyer follows the MERN stack architecture, consisting of four primary technologies:

MongoDB

Stores users, documents, extracted text, document questions, legal topics, bookmarks, and search history.

Express.js

Provides the REST API, server-side routing, middleware, and request handling.

React.js

Builds the interactive frontend and user interfaces.

Node.js

Provides the server-side JavaScript runtime environment.

Overall System Flow

React Frontend → Express.js API → Node.js Backend → Mongoose → MongoDB

For document-based question answering, the flow can be represented as:

User → React Interface → Q&A API → Document Retrieval → Relevant Passage Identification → Answer Generation → React Interface

When a user interacts with Pocket Lawyer, the React frontend sends requests to the Express.js backend.

Node.js processes the request and uses Mongoose to communicate with MongoDB.

For document questions, the backend retrieves the selected user's document, processes the relevant document content, and returns the answer through the API. The resulting data is returned through the API and rendered by the React interface.

This architecture provides a clear separation between the presentation layer, application logic, document processing, and data layer, making Pocket Lawyer easier to develop, maintain, test, and extend.

MODULES

Module 1: User Authentication & Management

This module handles user registration, login, and account management within the Pocket Lawyer platform. Users can create an account and log in to access their personalized legal document workspace.

Authentication provides the foundation for associating user activities with individual accounts. Once authenticated, users can upload documents, view their documents, ask questions, and access their personal history.

JWT-based authentication and password hashing can be used to secure user accounts and protected routes. The authentication system ensures that user-specific documents and private operations remain separated between different accounts.

Module 2: Document Upload & Management

The Document Upload and Management module allows users to upload and manage their legal documents.

Users can upload supported file formats including:

- PDF
- DOC
- DOCX
- TXT

The maximum supported file size is 10 MB. Uploaded documents are associated with the authenticated user and stored with relevant document information.

Users can view their uploaded documents and select a particular document for further interaction. This module forms the foundation of the document-based question-answering workflow.

Module 3: Document Text Extraction

The Document Text Extraction module processes uploaded files and extracts their readable text. After a document is uploaded, the backend processes the file and extracts its contents. The extracted text is stored along with the document record in MongoDB.

This extracted information is later used by the question-answering system to identify relevant sections of the document.

The module separates file processing from the frontend interface, allowing the React application to focus on user interaction while the backend handles document processing.

Module 4: Document-Based Question Answering

The Document-Based Question Answering module provides the primary intelligent feature of Pocket Lawyer. Users can select an uploaded document and ask questions related to its content.

The backend receives the document identifier and question and retrieves the corresponding document. Relevant passages are identified from the stored document text. The retrieved information is then used to generate a focused answer.

The result is returned to the React frontend and displayed to the user. Questions and answers can also be stored in MongoDB, allowing users to review their previous interactions with the document. This module connects document processing with an interactive question-answering workflow.

Module 5: Legal Topics & Search

The Legal Topics and Search module provides users with access to structured legal information. Users can browse or search available legal topics through the application.

This module complements document-based question answering by allowing users to explore legal concepts separately from their uploaded documents. Legal topic information is stored in MongoDB and can be retrieved through backend APIs. The React frontend displays the available topics in an organized interface.

Module 6: Bookmarks & Search History

The Bookmarks and Search History module helps users organize their interactions with the application.

Users can save useful legal topics or information through bookmarks. Search history maintains records of previous searches performed by the user. This information is associated with the user's account and stored in MongoDB.

The module allows users to revisit previously accessed information without repeating the same search.

Module 7: User Dashboard & Document Workspace

The User Dashboard provides a centralized view of the user's Pocket Lawyer activity.

The dashboard can display:

- Uploaded documents
- Recent questions
- Legal topics
- Bookmarked information
- Search history
- Document actions

The dashboard connects the major features of the application into a single workspace.

Users can upload a document, select an existing document, ask questions, and review their previous interactions through the dashboard. This module provides a central interface for managing the user's legal document activities.

Module Integration:

The seven modules work together as a connected system rather than as independent features.

The overall workflow can be represented as:

User Authentication → Upload Document → Extract Text → Select Document → Ask Question → Retrieve Relevant Content → Generate Answer → Save Q&A / History

The Authentication module establishes the user's identity.

The Document Management module handles uploaded documents.

The Text Extraction module prepares document content for processing.

The Q&A module uses the extracted document content to answer user questions.

The Legal Topics module provides additional structured legal information.

Bookmarks and Search History allow users to save and revisit useful information.

The Dashboard connects these features into a single user workspace.

All these modules are connected through the MERN architecture, with React handling user interaction, Node.js and Express.js providing backend APIs, and MongoDB with Mongoose handling persistent data storage.

FRONTEND CODE

Application Entry & Authentication

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import { AuthProvider } from '@/context/AuthContext';
import './index.css';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </React.StrictMode>
);
```

Application Routing & Navigation

```
export default function App() {
  return (
    <BrowserRouter>
      <Toaster position="bottom-right" />

      <AppShell>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/explore" element={<Explore />} />
          <Route path="/search" element={<SearchPage />} />
          <Route path="/topic/:id" element={<TopicDetail />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/documents" element={<Documents />} />
          <Route path="/documents/:id" element={<DocumentDetail />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
        </Routes>
      </AppShell>
    </BrowserRouter>
```

```
  );
 }
```

User Authentication Management:

```
 const AuthContext = createContext(null);

 export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] =
   useState(Boolean(getToken()));

  const login = async (values) => {
   const data = await api.post("/auth/login", values);
   setToken(data.token);
   setUser(data.user);
   return data.user;
  };

  const register = async (values) => {
   const data = await api.post("/auth/register", values);
   setToken(data.token);
   setUser(data.user);
   return data.user;
  };

  const logout = () => {
   setToken(null);
   setUser(null);
  };

  return (
   <AuthContext.Provider
    value={{
     user,
     loading,
     isAuthenticated: Boolean(user),
     login,
     register,
     logout
    }}
   >
    {children}
   </AuthContext.Provider>
  );
}
```

Common Layout & Navigation:

```
export function AppShell({ children }) {
  const [location] = useLocation();
  const [mobileOpen, setMobileOpen] = useState(false);

  const { user, isAuthenticated, logout } = useAuth();

  const links = [
    ["/", "Home"],
    ["/explore", "Explore"],
    ["/search", "Search"],
    ["/about", "About"]
  ];

  return (
    <div className="app-shell">
      <header className="site-header">
        {/* Navigation and authentication */}
      </header>

      <main>{children}</main>

      <footer className="site-footer">
        {/* Footer links and information */}
      </footer>
    </div>
  );
  }
```

Legal Topic Display

```
export function TopicCard({ topic }) {
  return (
    <Link
      href={`/topic/${topic._id || topic.slug}`}
      className="topic-card"
    >
      <span>{topic.category}</span>
      <h3>{topic.title}</h3>
      <p>{topic.description}</p>

      <div className="tag-row">
        {(topic.tags || [])
          .slice(0, 3)
          .map((tag) => (
            <span className="tag" key={tag}>
```

```
{tag}
            </span>
          ))}
      </div>
    </Link>
  );
}
```

TopicCard

```
export function TopicCard({ topic }) {
  return (
    <Link href={`/topic/${topic._id || topic.slug}`}
      className="topic-card">
      <span>{topic.category}</span>
      <h3>{topic.title}</h3>
      <p>{topic.description}</p>
      <div className="tag-row">
        {(topic.tags || [])
          .slice(0, 3)
          .map((tag) => (
            <span className="tag" key={tag}>
              {tag}
            </span>
          ))}
      </div>
    </Link>
  );
}
```

BACKEND CODE

Server:

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { createServer } from 'node:http';
import { connectDB } from './server/config/db.js';
import apiRouter from './server/routes/index.js';
import { errorMiddleware } from './server/middleware/errorMiddleware.js';
const app = express();
app.use(cors({ origin: process.env.CLIENT_ORIGIN || true }));
app.use(express.json({ limit: '2mb' }));
app.use('/uploads', express.static('server/uploads'));
app.use('/api', apiRouter);
app.use(errorMiddleware);
const port = Number(process.env.PORT || 3000);
await connectDB();
createServer(app).listen(port, '0.0.0.0',
  () => console.log(`Server running on port ${port}`)
);
```

Authentication:

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';
export async function authMiddleware(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ')
      ? header.slice(7)
      : req.cookies?.token;
    if (!token)
      return res.status(401).json({
        message: 'Authentication required.'
      });
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(payload.id);
```

```
    if (!user)
      return res.status(401).json({
        message: 'Session is no longer valid.'
      });
    req.user = user;
    next();
  } catch {
    res.status(401).json({
      message: 'Authentication required.'
    });
  }
}
```

Document Upload & Text Extraction

```
export async function uploadDocument(req, res) {
  if (!req.file || !allowed.has(req.file.mimetype))
    return res.status(400).json({
      message: 'Upload a PDF, DOC, DOCX, or TXT file.'
    });

  const doc = await Document.create({
    userId: req.user._id,
    fileName: req.file.originalname,
    fileType: req.file.mimetype,
    filePath: req.file.path,
    status: 'processing'
  });

  try {
    const extractedText = await extractText(
      req.file.path,
      req.file.mimetype,
      req.file.originalname
    );

    doc.extractedText = extractedText.slice(0, 2000000);
    doc.status = 'ready';

    await doc.save();
    res.status(201).json(doc);
  } catch {
    doc.status = 'failed';
    await doc.save();
```

```
 res.status(422).json({
      message: 'Document processing failed.'

    });
  }
}
```

Document-Based Q&A:

```
export async function askQuestion(req, res) {
  const question = String(req.body.question || '').trim();

  if (question.length < 3)
    return res.status(400).json({
      message: 'Ask a more specific question.'
    });

  const doc = await Document.findOne({
    _id: req.params.id,
    userId: req.user._id
  });

  if (!doc)
    return res.status(404).json({
      message: 'Document not found.'
    });

  if (doc.status !== 'ready')
    return res.status(409).json({
      message: 'This document is not ready for questions.'
    });

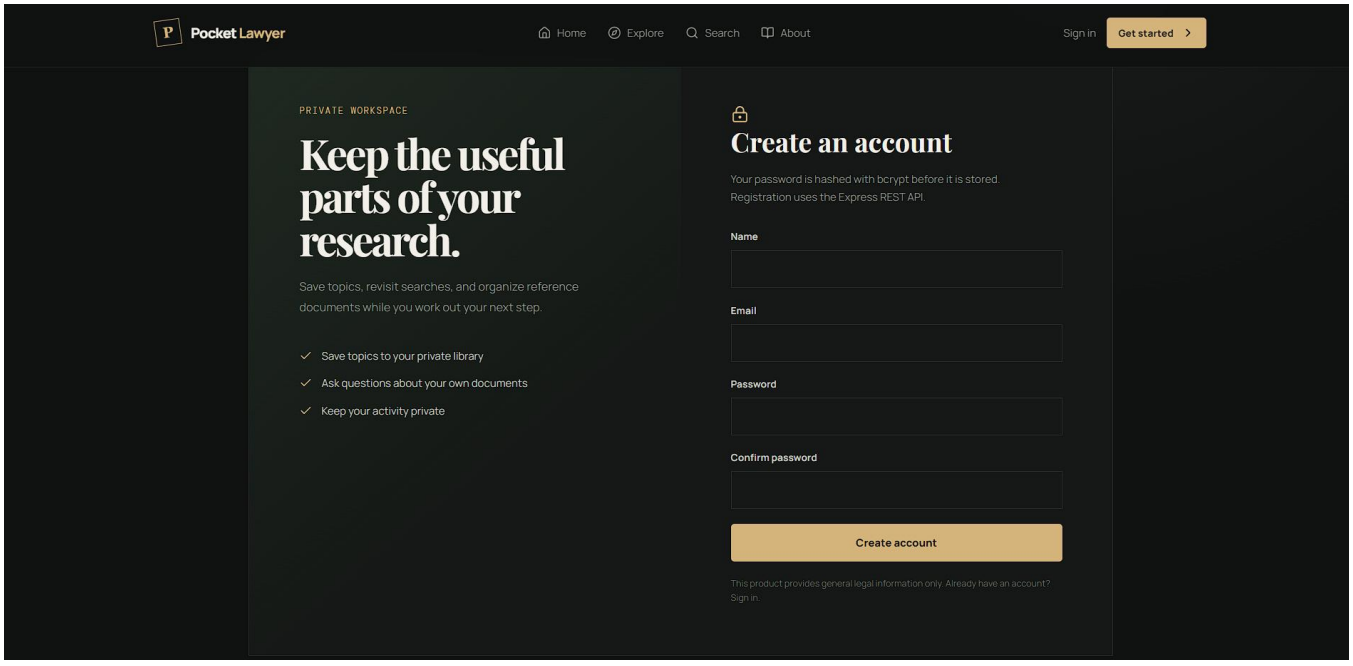
  const result =
    answerFromDocument(question, doc.extractedText);

  const saved = await DocumentQuestion.create({
    documentId: doc._id,
    userId: req.user._id,
    question,
    ...result
  });

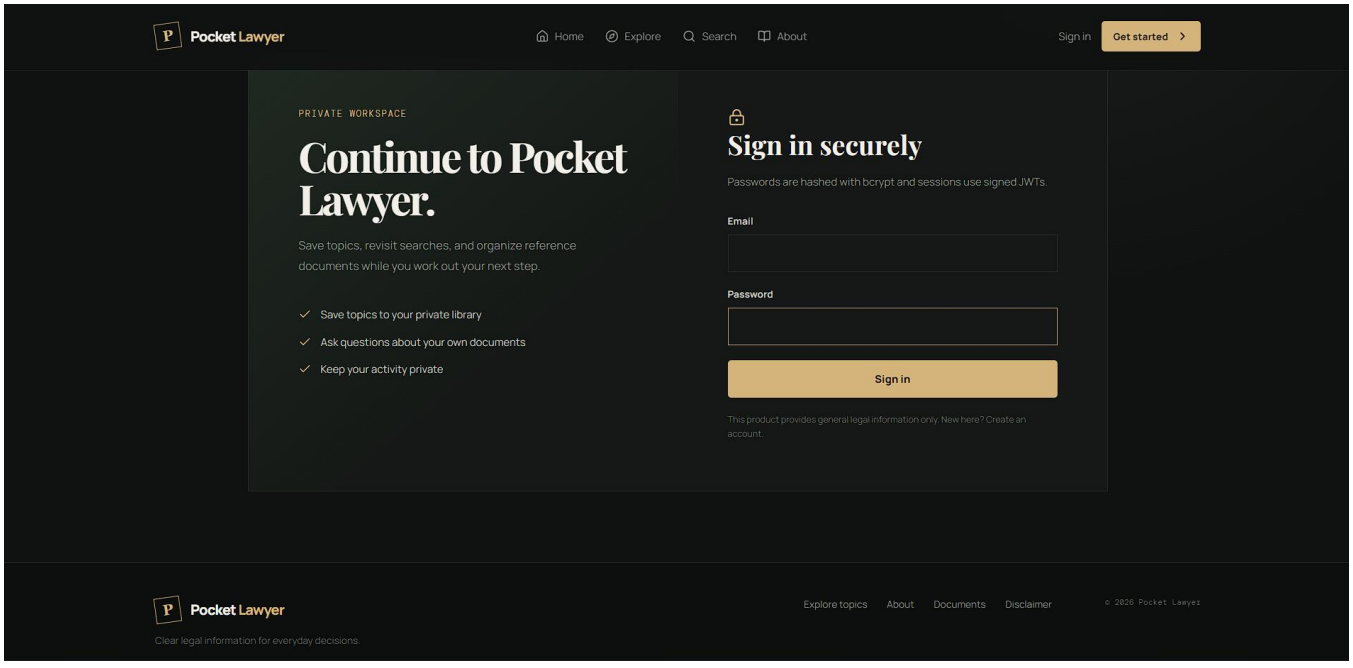
  res.status(201).json(saved);
}
```

OUTPUT

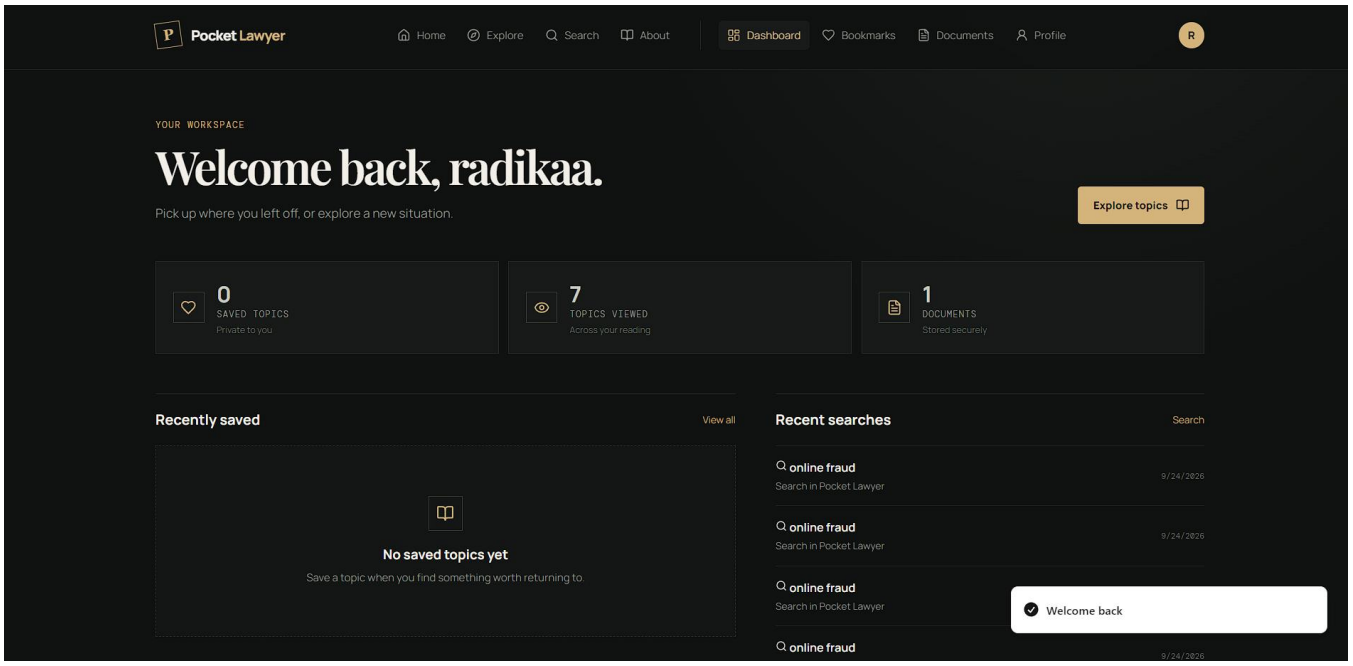
Register Page:



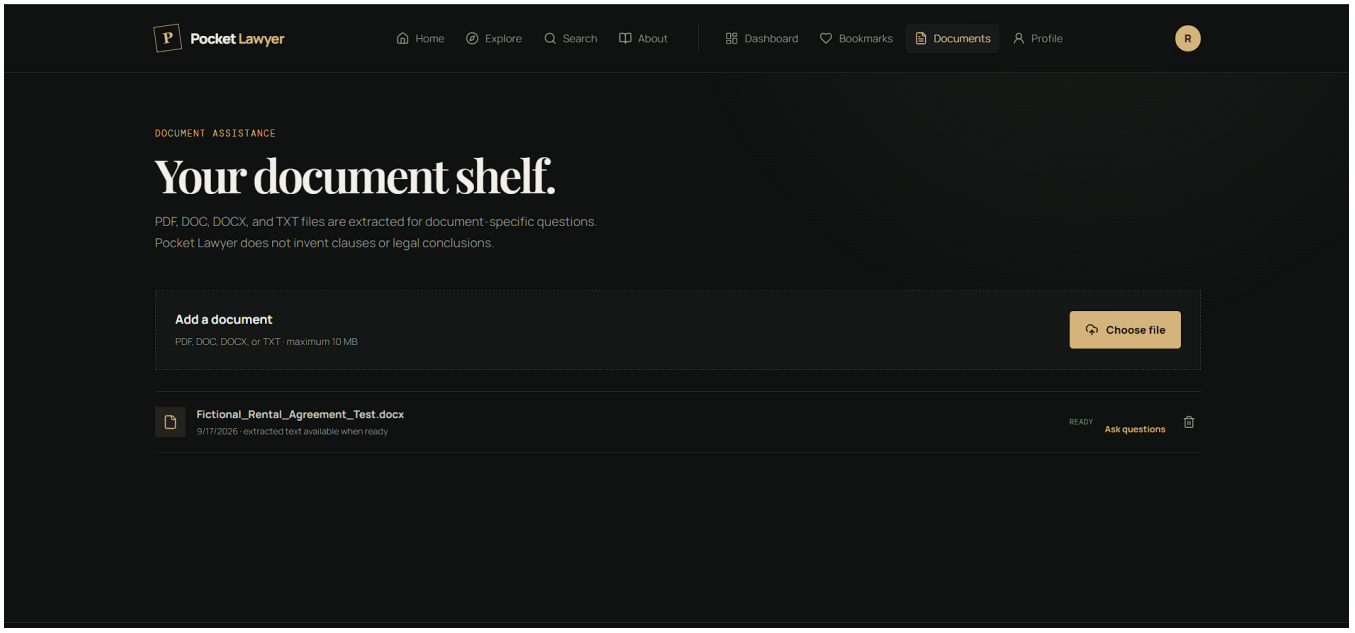
Login Page:



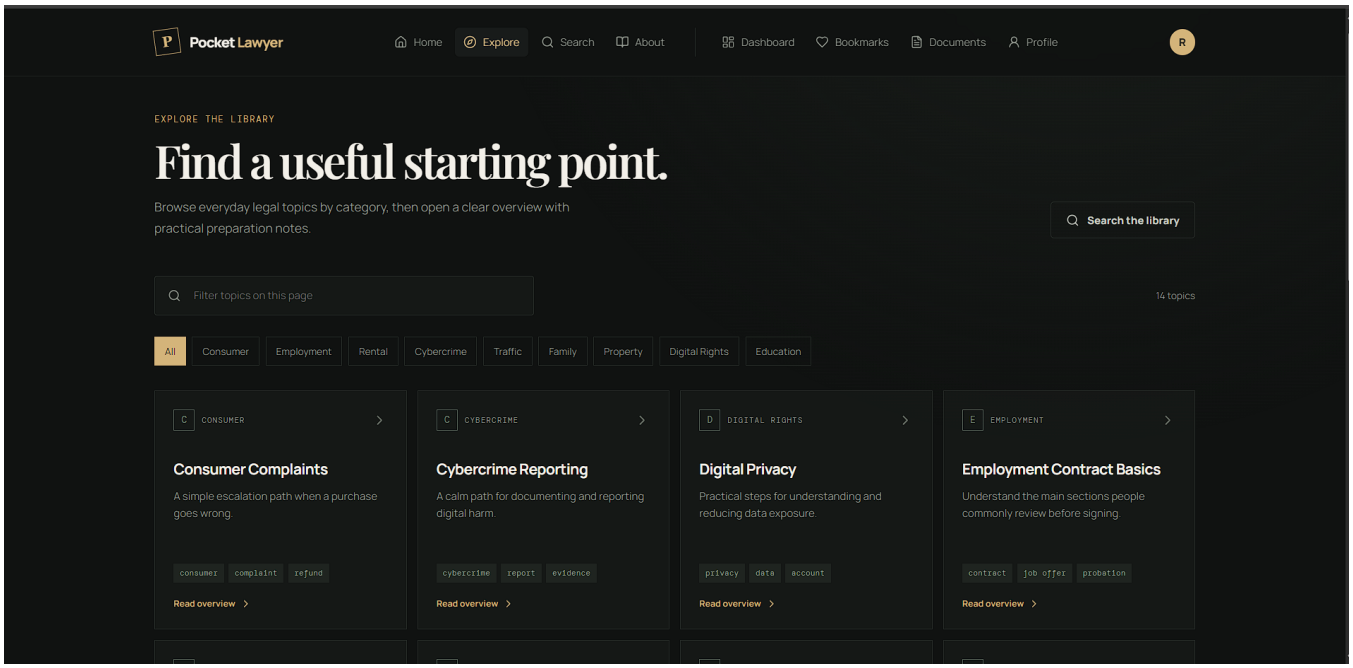
Dashboard:



Document-Based Q&A:



Topics:


Pocket Lawyer
Home
Explore
Search
About
Dashboard
Bookmarks
Documents
Profile
R
EXPLORE THE LIBRARY
Find a useful starting point.
Browse everyday legal topics by category, then open a clear overview with practical preparation notes.
Search the library
Filter topics on this page
14 topics
All
Consumer
Employment
Rental
Cybercrime
Traffic
Family
Property
Digital Rights
Education
C CONSUMER
Consumer Complaints
A simple escalation path when a purchase goes wrong.
consumer complaint refund
Read overview
C CYBERCRIME
Cybercrime Reporting
A calm path for documenting and reporting digital harm.
cybercrime report evidence
Read overview
D DIGITAL RIGHTS
Digital Privacy
Practical steps for understanding and reducing data exposure.
privacy data account
Read overview
E EMPLOYMENT
Employment Contract Basics
Understand the main sections people commonly review before signing.
contract job offer probation
Read overview